

Design Pattern Report: Interpreter Pattern

Group 52

Nguyễn Đình Minh Huy (Student ID: 25125083)

Trần Gia Huy (Student ID: 25125084)

Ngày 18 tháng 8 năm 2026

Mục lục

1	Real-world Problem	2
2	Naive Solution (Without Design Pattern)	2
3	Problems with the Naive Solution	2
4	Introduction to the Interpreter Pattern	2
5	General Class Diagram	3
6	Class Diagram for the Logic Evaluator	3
7	Implementation of the Interpreter Pattern	4
8	Pros and Cons	4
8.1	Pros	4
8.2	Cons	4
9	Other Real-world Applications	5
9.1	Web Development	5
9.2	Mobile Development	5
10	References	5

1 Real-world Problem

In many systems, there is a need to evaluate language expressions or logical constraints defined dynamically at runtime. For example, database query parsers, business rule engines, or search filters. Consider a logic engine that needs to evaluate Boolean expressions containing variables (e.g., A , B , C) and logical operations (AND, OR, NOT). This can be used in evaluating if-else statements or Boolean satisfiability problem (B-SAT). The engine must accept various combinations of expressions dynamically, lookup the values of variables in a configuration context, and return the final truth value.

2 Naive Solution (Without Design Pattern)

In a naive C++ implementation, expressions are represented as raw strings. The evaluation is performed by string searching and direct logical checks.

```
1 // Refer to Interpreter/src/naive.cpp for full implementation details
2 class NaiveLogicEvaluator {
3 public:
4     static bool evaluate(const std::string& expression, const
      NaiveContext& context) {
5         if (expression.find(" AND ") != std::string::npos) {
6             size_t pos = expression.find(" AND ");
7             std::string left = expression.substr(0, pos);
8             std::string right = expression.substr(pos + 5);
9             return context.lookup(left) && context.lookup(right);
10        }
11        // Similar blocks for OR and NOT
12        ...
13    }
14 };
```

Listing 1: Naive Boolean Expression Evaluator

3 Problems with the Naive Solution

The naive string-parsing solution has significant limitations:

1. **Violate OCP:** When adding a new operation or a terminal expression, we have to change logic of the evaluate or interpret function. This violates Open-Close Principle, which makes the application harder to scale or change the structure.
2. **Complexity of nested expression:** Evaluate a nested expression such as $(A \text{ AND } B) \text{ OR } (\text{NOT } C \text{ AND } D)$ with string searching becomes complex, error-prone and slow.
3. **High coupling:** Parsing the syntax and evaluating occurs at the same function. This violates Single Responsibility Principle (SRP).

4 Introduction to the Interpreter Pattern

The **Interpreter Pattern** is a behavioral design pattern that defines a grammatical representation for a language along with an interpreter to evaluate sentences in the language.

It represents grammar rules as classes. These classes can be structured hierarchically to form an Abstract Syntax Tree (AST).

Key elements:

- **Context:** Contains information global to the interpreter (e.g., variable mappings).
- **AbstractExpression:** Declares an abstract `interpret(Context)` operation.
- **TerminalExpression:** Implements an `interpret` operation associated with terminal symbols in the grammar (e.g., variables, constants).
- **NonterminalExpression:** Implements an `interpret` operation for nonterminal symbols (e.g., operators like `AndExpression`, `OrExpression`).

5 General Class Diagram

In the general Interpreter pattern:

- **Client** builds the AST out of Terminal and Nonterminal expressions.
- The root of the tree is evaluated by calling `interpret()` with a given **Context**.

[Insert General Interpreter UML Diagram Here]

Hình 1: General Interpreter Design Pattern UML Diagram

6 Class Diagram for the Logic Evaluator

For our Boolean expression evaluator:

- **Expression** is the abstract base.
- **VariableExpression** and **ConstantExpression** are terminal expressions.
- **AndExpression**, **OrExpression**, **NotExpression**, and **XorExpression** are nonterminal expressions.

[Insert Logic Interpreter UML Diagram Here]

Hình 2: UML Class Diagram for Boolean Logic Interpreter

7 Implementation of the Interpreter Pattern

By modeling rules as classes, expressions are composed as trees.

```
1 // Refer to Interpreter/src/pattern.cpp for full implementation details
2 class Expression {
3 public:
4     virtual ~Expression() = default;
5     virtual bool interpret(const Context& context) = 0;
6 };
7
8 class VariableExpression : public Expression {
9     std::string name;
10 public:
11     VariableExpression(std::string name) : name(name) {}
12     bool interpret(const Context& c) override { return c.lookup(name);
13 }
14 };
15 class AndExpression : public Expression {
16     // std::unique_ptr enforces a strict Abstract Syntax Tree (AST)
17     // tree hierarchy,
18     // ensuring exclusive single ownership for each child node.
19     std::unique_ptr<Expression> left, right;
20 public:
21     AndExpression(std::unique_ptr<Expression> l, std::unique_ptr<
22 Expression> r)
23         : left(std::move(l)), right(std::move(r)) {}
24     bool interpret(const Context& c) override {
25         return left->interpret(c) && right->interpret(c);
26     }
27 };
```

Listing 2: Interpreter Pattern Implementation

8 Pros and Cons

8.1 Pros

- **Easy to Change/Extend Grammar:** Since rules are represented by classes, you can use inheritance to change or extend the grammar easily (e.g., adding `XorExpression`).
- **Simple Implementation:** Classes defining nodes in the AST are straightforward to write.

8.2 Cons

- **Complex Grammars are Hard to Maintain:** As the grammar rules increase, the class hierarchy becomes large and difficult to manage. For complex grammars, compiler generators (like ANTLR, Lex/Yacc) are preferred.
- **Efficiency:** Building and traversing ASTs can be slower and memory-intensive compared to state-machine-based parsers.

9 Other Real-world Applications

9.1 Web Development

- **URL Routing Parsers:** Routers in frameworks (e.g., Express.js, Django) parse URL paths matching dynamic patterns like `/user/:id/profile`.
- **CSS Selector Engines:** Browsers interpret selector strings like `div.container > ul li` to apply styles.

9.2 Mobile Development

- **Localization/Pluralization Parsers:** Processing format strings containing plural conditions (e.g., "1 item" vs. "many items").
- **Formula Evaluators:** Spreadsheet apps or calculator features on mobile apps.

10 References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns*. Addison-Wesley.
- Refactoring.Guru. *Design Patterns: Interpreter*. <https://refactoring.guru/design-patterns/interpreter>